# Zamp Invoice Intelligence

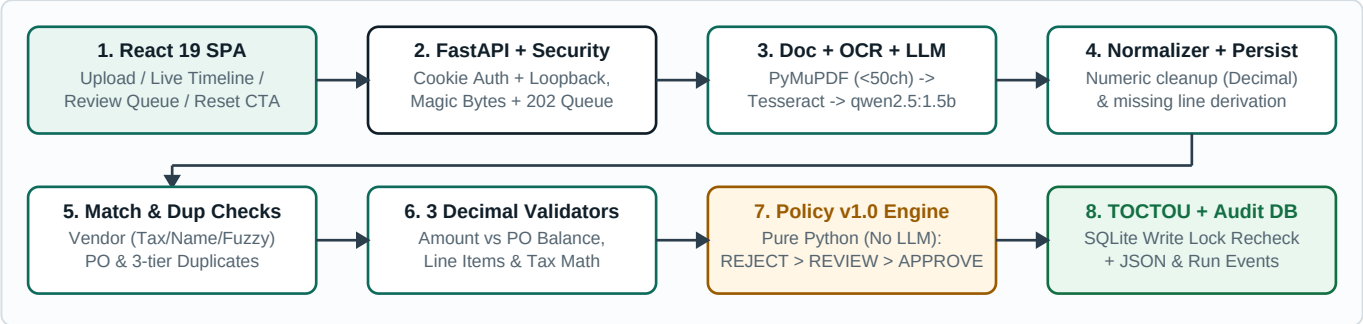
Complete System Architecture, Implementation Reference & 5-Minute Live Working Demo Playbook

**Local Stack:** http://127.0.0.1:8000 · **LLM:** Ollama (qwen2.5:1.5b) · **OCR:** Tesseract v5.4 · **Policy:** v1.0

| FRONTEND & DELIVERY | BACKEND & STORAGE | DOCUMENT AI LAYER | GOVERNANCE & SAFETY |
|---|---|---|---|
| **React 19 + Vite SPA** | **FastAPI + SQLite** | **PyMuPDF + Ollama** | **Policy v1.0 Engine** |
| TypeScript + React Router v6 built to static assets served directly by FastAPI on :8000. | SQLAlchemy 2.0 ORM, strict Decimal currency arithmetic, 7 relational tables + JSON snapshots. | Digital PDF parser (>=50 chars/pg), Tesseract OCR fallback, local qwen2.5:1.5b JSON extractor. | 100% deterministic rules (REJECT > REVIEW > APPROVE), TOCTOU re-check, Reset & Retry controls. |

## 1. Problem Statement & End-to-End Architecture

Accounts-payable teams must reconcile incoming PDF vendor invoices against procurement Purchase Orders (POs) before releasing funds. Manual verification is slow and error-prone, while letting an LLM directly approve payments creates unacceptable hallucination and audit risk. **Zamp Invoice Intelligence** solves this with a strict architectural boundary: **AI extracts unstructured text into schema-validated fields, while deterministic Python code matches, validates arithmetic, evaluates policy v1.0, and writes an immutable stage-by-stage audit log.**




### Core Engineering Guarantees (High-Value Interview Talking Points)

- **Zero LLM Decision Authority:** The LLM only populates `InvoiceExtraction` JSON with page-grounded evidence. `decide()` in `decision_engine.py` never calls any model and produces identical outcomes on identical inputs.
- **Strict Financial Arithmetic:** All monetary amounts, tax totals, and PO balances run through Python `Decimal` — never IEEE floats — preventing penny rounding drift across cumulative partial invoices.
- **Honest Failure Separation:** Technical crashes (e.g., Ollama unreachable or invalid PDF) transition a run to `FAILED` with `decision=None` and preserve prior events so operators can click **Retry**; failed runs never masquerade as business decisions.
- **Fail-Safe Uncertainty Escalation:** Missing fields, ambiguous candidates, provisional balances, low model confidence (<0.80), or even an unexpected validator error route straight to **REVIEW**. Only fully clean invoices ever auto-**APPROVE**.
- **Real Persisted Live Timeline:** Every stage commits a row to `run_events`; the React frontend polls `GET /api/runs/{run_id}` every ~2 seconds rather than animating a fake progress bar.

**Active Windows Runtime Verified Today**

FastAPI + built React SPA runs on **http://127.0.0.1:8000** using `backend/data/demo.db` (keeping development evidence `app.db` untouched). Local Ollama serves **qwen2.5:1.5b** on **http://127.0.0.1:11434** via `LLM_PROVIDER=llamacpp`, Tesseract OCR v5.4 handles rasterized pages, and test suites pass (**281 backend pytest** + **52 frontend Vitest**).

## 2. Relational Data Model, Seeded Baseline & Security Controls

### SQLite / SQLAlchemy Schema (7 Tables in `backend/app/db/models.py`)

| Table | Responsibility & Key Columns | Layer |
|---|---|---|
| `vendors` | Vendor master records (`V-001..V-006`): legal & normalized name, GST/tax ID, status (`ACTIVE \| BLOCKED \| INACTIVE`). | Master |
| `purchase_orders` | PO headers (`PO-2041..PO-2048`) linked to vendor: issue date, currency (`INR`), total amount, `OPEN \| CLOSED \| CANCELLED`. | Master |
| `purchase_order_lines` | Authorized line items per PO: description, SKU, quantity (`Numeric(12,4)`), unit price, line total. | Master |
| `invoices` | Submitted invoices + extracted financials, server-generated `storage_path`, `original_filename` (display only), `file_sha256`. | Runtime |
| `invoice_lines` | Extracted line items for an invoice; cleared & rewritten atomically on re-runs/retries to prevent line duplication. | Runtime |
| `processing_runs` | UUID-keyed execution record: `status`, `decision`, LLM provider/model, plus **9 persisted JSON audit columns**. | Audit |
| `run_events` | Chronological per-stage events (`stage, status, message, details_json, created_at`) driving the live timeline. | Audit |

### Seeded Procurement Baseline (`seed.py` — Restored by Reset)

| PO # | Vendor | PO Total | Status | Demo / Test Significance |
|---|---|---|---|---|
| PO-2041 | V-001 ABC Industrial Supplies | 100,000.00 INR | **OPEN** | Has pre-seeded INV-2026-1042 (59,000, unapproved) |
| PO-2042 | V-002 XYZ Metals and Alloys | 75,000.00 INR | **OPEN** | Has pre-seeded INV-2026-1043 (47,200, unapproved) |
| PO-2043 | V-003 Delta Supply Chain | 250,000.00 INR | **CLOSED** | Closed PO test target (no tax_id on vendor) |
| PO-2044 | V-001 ABC Industrial Supplies | 60,000.00 INR | **OPEN** | **Clean Approve demo target** (Welding + Goggles) |
| PO-2045 | V-002 XYZ Metals and Alloys | 120,000.00 INR | **OPEN** | **Overbilling edge case target** (4 PO lines) |
| PO-2046 | V-005 Golden Hardware Traders | 45,000.00 INR | **OPEN** | **OCR + Provisional target** (has unapproved INV-2026-1044) |
| PO-2047 | V-004 Sharma Electrical | 30,000.00 INR | **BLOCKED** | **Blocked vendor REJECT target** (V-004 BLOCKED) |
| PO-2048 | V-006 Pune Precision Eng. | 85,000.00 INR | **INACTIVE** | Inactive vendor & closed PO edge test |

*Note:* Reset restores **6 vendors, 8 POs (21 PO lines), 3 unrun sample invoices** (`INV-2026-1042..1044`), and **0 processing runs**.

### Security, Auth, Reset & Concurrency Protections

| Control | Implementation Behavior |
|---|---|
| Session & Bearer Auth | `auth_guard` (`main.py`, `auth.py`): When `AUTH_PASSWORD` is set, all `/api/*` endpoints except auth status/login require a 12h HMAC-SHA256 `HttpOnly; SameSite=Strict` cookie (`zamp_session`) or `Bearer` header. |
| Write Guard & Rate Limits | `require_write_access` (`security.py`): `/upload`, `/retry`, `/dashboard/reset`, and diagnostics require `X-Zamp-Token` or loopback IP; sliding-window limiter caps uploads (30/min), diagnostics (10/min), and logins (5/min). |
| Upload Path Confinement | `save_uploaded_pdf` & `validate_pdf`: Verifies `.pdf` extension, non-empty, <=20 MB, and `%PDF-` magic header. Stores files as `<16hex>_<sha16>.pdf` under `UPLOAD_DIR`; document downloads resolve strictly inside `UPLOAD_DIR`. |
| HTTP Security Headers | Middleware adds `Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and `Referrer-Policy` on all responses. |
| Workspace Reset Guard | `POST /api/dashboard/reset` (`reset_demo_data`): Acquires thread lock, rejects with **409 Conflict** if any run is `PENDING` or `RUNNING`, deletes DB records + scoped upload PDFs, and re-seeds atomically. |

## 3. End-to-End 11-Stage Execution Workflow (`workflow.py`)

When a user clicks **Upload and process**, `POST /api/invoices/upload` saves the file, creates an `Invoice` + `ProcessingRun(PENDING)` + initial `RECEIVED` event, schedules `process_invoice_background` via FastAPI `BackgroundTasks`, and returns **202 Accepted** immediately.

| Stage | Service / Function | Exact Execution & Failure-Safety Behavior |
|---|---|---|
| 0. Atomic Claim | `_claim_run` | Executes atomic `UPDATE ... WHERE id=:id AND status='PENDING'` to transition `PENDING -> RUNNING`. Prevents duplicate processing if retried/raced. |
| 1. PDF Text | `extract_pdf` | PyMuPDF reads page text. Pages with >=50 chars use `PDF_TEXT`; pages below 50 chars are tagged `OCR_REQUIRED` with warnings. |
| 2. OCR Fallback | `OCRService.ocr_page` | Skipped (`SKIPPED`) on digital PDFs. On scanned pages, renders 2x matrix PNG (~144 DPI) -> Tesseract `eng`; clears stale 'no extractable text' warnings when recovered. |
| 3. LLM Extraction | `extract_invoice_from_text` | **Commits DB session first** so slow inference never locks SQLite! Uses `COMPACT_INVOICE_EXTRACTION_PROMPT` for `llamacpp`, cleans currency glyphs (`_clean_number`), parses strict Pydantic decimals. |
| 3b. Line Persist | `_persist_extraction` | Deletes prior lines (retry-safe). Deterministically computes a missing 3rd quantity/unit_price/line_total if the other 2 exist; skips incomplete lines with a warning rather than guessing. |
| 4. Vendor Match | `match_vendor` | Cascade: Exact Tax ID -> Exact normalized name -> RapidFuzz `token_sort_ratio >= 90`. Sets `inv.vendor_id`. Failure is non-fatal (surfaces in `unknown_checks`). |
| 5. PO Match | `match_po` | Exact normalized PO number (verifies vendor ownership) or candidate search on matched vendor's OPEN/CLOSED POs. Sets `inv.purchase_order_id`. |
| 6. Duplicate Check | `detect_duplicates` | Passes `exclude_invoice_id=inv.id`. Checks (A) SHA-256 bytes, (B) vendor + invoice #, (C) RapidFuzz similarity >=85 + financial match. |
| 7. Amount Check | `validate_amount` | Decimal subtraction against PO balance committed by **completed APPROVE runs only**. Checks currency mismatch, +/-100.00 INR tolerance, and provisional history. |
| 8. Line Items | `validate_line_items` | Pairs invoice lines vs PO lines (Exact SKU -> normalized desc -> RapidFuzz >=85). Verifies quantities, unit prices, and unbilled/extra items. |
| 9. Tax / Math | `validate_tax_arithmetic` | Reconciles `abs((subtotal + tax - discount) - total) <= 0.01` using `Decimal`. Missing subtotal/total yields `UNKNOWN`. |
| 10. Decision | `decide` | Pure deterministic policy v1.0 evaluation (`decision_engine.py`). No LLM call, no DB writes. Produces `REJECT \| REVIEW \| APPROVE` + rules. |
| 11. Persistence + TOCTOU | `_stage_persistence` | Before writing `APPROVE`, acquires SQLite `RESERVED` lock (`UPDATE purchase_orders SET id=id`) and re-validates balance; downgrades stale `APPROVE` to `REVIEW (AMOUNT_003)`. Writes all 9 JSON columns + `COMPLETED`. |

**Resilience Highlight 1: Real-World Invoice Number Cleanup & Local LLM Prompt Strategy**

In `llm_extractor.py`, `_clean_number()` strips Indian/Western thousands commas (`1,00,000.00`), currency symbols (Rupee symbol, `INR`), `'null'` strings, and common OCR symbol glitches (e.g., `I60000.00 -> 60000.00`) *before* strict Pydantic Decimal validation. Meanwhile, `extraction_prompt.py` automatically switches between `INVOICE_EXTRACTION_PROMPT` (cloud OpenRouter) and `COMPACT_INVOICE_EXTRACTION_PROMPT` (local `llamacpp` 1.5B models) so quantized models retain `invoice_number` and per-field confidence evidence cleanly.

**Resilience Highlight 2: Concurrency Race Protection (TOCTOU Guard)**

If two invoices targeting the same PO arrive simultaneously, Stage 7 validates both against a point-in-time balance snapshot. Before persisting an `APPROVE` in Stage 11, `_stage_persistence` takes a SQLite write lock on that PO row and re-runs `validate_amount()`. If the first worker already committed and depleted the balance, the second worker immediately downgrades from **APPROVE** to **REVIEW** via rule `AMOUNT_003`, preventing double-billing.

## 4. Deterministic Validators & Policy v1.0 Decision Engine

Every applicable rule is evaluated across Phase A (Hard Reject), Phase B (Human Review), and Phase C (Auto-Approve). Precedence is strictly **REJECT > REVIEW > APPROVE** — a passing financial check can never override a blocked vendor or duplicate hash, and `approval_eligible` requires every mandatory check to pass with extraction confidence **>= 0.80**.

| Outcome | Rule ID | Rule Name | Exact Trigger Condition (`decision_engine.py` / `decision_policy.py`) |
|---|---|---|---|
| **REJECT** | `DOC_DUP_001` | Exact document duplicate | Same PDF SHA-256 bytes already exist on another invoice (`exclude_invoice_id` prevents self-match). Stops renamed duplicate uploads. |
| **REJECT** | `INV_DUP_001` | Exact invoice duplicate | Same vendor + normalized invoice number AND matching total/currency (prevent double entry). |
| **REJECT** | `VENDOR_BLOCK_001` | Blocked vendor (exact) | Primary vendor matched cleanly via Tax ID or exact normalized name has status `BLOCKED`. Fuzzy-only hits never auto-reject. |
| **REVIEW** | `INV_DUP_002` | Duplicate w/ diff financials | Same vendor + invoice # as prior record, but differing total_amount or currency. |
| **REVIEW** | `DUP_001` | Potential duplicate | Same vendor, similar invoice # (RapidFuzz >= 85), and matching total or currency. |
| **REVIEW** | `FIELDS_001` | Missing critical fields | Any of `invoice_number`, `vendor_name`, or `total_amount` is null/blank. |
| **REVIEW** | `VENDOR_001..005` | Vendor exceptions | UNMATCHED (`001`), AMBIGUOUS candidates (`002`), NEEDS_REVIEW (`003`), INACTIVE (`004`), or Fuzzy < 0.95 (`005`). |
| **REVIEW** | `PO_001..008` | PO exceptions | MISSING_PO (`001`), AMBIGUOUS / vendor mismatch (`002`), UNMATCHED (`003`), CANCELLED (`004`), CLOSED (`005`), Fuzzy (`006`), not OPEN (`007`), bare candidate without PO ref (`008`). |
| **REVIEW** | `CURRENCY_001` | Currency mismatch | Invoice currency differs from matched PO currency — comparison suspended. |
| **REVIEW** | `AMOUNT_001` | PO balance exceeded | Invoice exceeds remaining committed PO balance by more than `100.00` absolute tolerance. |
| **REVIEW** | `AMOUNT_002` | Provisional PO balance | No completed `APPROVE` history exists on this PO, yet unapproved invoices already reference it. |
| **REVIEW** | `AMOUNT_003` | TOCTOU balance drift | Stage 11 persistence lock re-check detected concurrent PO balance exhaustion mid-flight. |
| **REVIEW** | `LINEITEM_001` | Line-item discrepancy | Qty or unit price mismatch (outside strict `Decimal('0.01')` diff check) or unmatched invoice/PO lines. |
| **REVIEW** | `TAX_001` / `TAX_002` | Tax / math issue | `TAX_001`: `\|subtotal + tax - discount - total\| > 0.01`. `TAX_002`: subtotal or total missing. |
| **REVIEW** | `EXTRACTION_001` | Low extraction quality | Extraction status not `SUCCESS`, non-empty warnings, or `min_extraction_confidence < 0.80` (or omitted -> fails eligibility). |
| **REVIEW** | `CHECK_001` | Unknown check | One or more Stage 4–9 validators returned `UNKNOWN` or raised an exception. |
| **APPROVE** | `APPROVE_001` | All checks passed | Active exact vendor, exact OPEN PO, no duplicates, PASS on amount/lines/tax, SUCCESS extraction with min confidence >= 0.80. |

**Important Nuance for Live Demos with Local Small Models (`qwen2.5:1.5b`)**

**1. Why might a clean digital invoice occasionally land in REVIEW instead of APPROVE?**
In `decision_engine.py` Line 619, auto-approval enforces `(min_extraction_confidence or 0) >= 0.80`. Our `COMPACT_INVOICE_EXTRACTION_PROMPT` instructs `qwen2.5:1.5b` to emit `{"text": ..., "page": 1, "confidence": 0.8}` (which passed on `syn_01_clean_approve.pdf` in live verification -> green **APPROVE**!). However, a 1.5B quantized model occasionally emits plain strings in `evidence`, making `min_extraction_confidence` evaluate to `None`. When that happens, policy v1.0 deliberately refuses to auto-approve un-evidenced extraction and falls back to **REVIEW** (`"mandatory conditions not fully satisfied"`). Turn that moment into a strength: explain fail-safe engineering!

# 5. Timed 5-Minute Live Working Demo Script (No Slides)

**Pre-Demo 60-Second Readiness Checklist (Run Before Audience Arrives)**

**1. Servers Running:** Keep Ollama (`http://127.0.0.1:11434`, model `qwen2.5:1.5b`) and Zamp FastAPI (`http://127.0.0.1:8000`) running. Verify `/health` returns `ok`.
**2. Browser Open & Signed In:** Open **http://127.0.0.1:8000** in Chrome/Edge; if prompted, enter password `ZampPhase9Demo!` (from `backend/.env`).
**3. File Picker Ready:** Open File Explorer / upload folder path once beforehand to `backend\data\synthetic_invoices` so selecting PDFs takes 2 seconds.
**4. Timing Budget:** On local CPU, `qwen2.5:1.5b` takes ~**13–18 seconds per invoice**. Uploading **3 PDFs** live takes ~45s compute total, leaving 4+ minutes for rich narration and UI inspection.

| Time | Demo Beat | Live Action | What Reviewer Sees |
|---|---|---|---|
| **0:00 – 0:45** (45s) | Problem Hook + Overview + Live Reset | Overview -> Click **Reset data** -> **Confirm reset** | Restores clean baseline (**3** seeded invoices, **0** runs). Shows live LLM badge. |
| **0:45 – 2:10** (85s) | Happy Path + Live 11-Stage Pipeline | Upload `syn_01_clean_approve.pdf` (~17s) | Watch live timeline poll every 2s -> Detail page -> **APPROVE** (`APPROVE_001`). |
| **2:10 – 3:35** (85s) | **Edge Case #1:** Overbilling & Qty Mismatch | Upload `syn_03_overbilling.pdf` (~15s) | All header/tax checks pass, yet **AMOUNT_001** + **LINEITEM_001** force **REVIEW**. |
| **3:35 – 4:25** (50s) | **Edge Case #2:** Blocked Vendor Precedence | Upload `syn_02_blocked_vendor.pdf` (~13s) | Exact PO + math match, yet **VENDOR_BLOCK_001** forces immediate **REJECT**. |
| **4:25 – 5:00** (35s) | Review Queue, History, Retry & Closing | Open **Review Queue** & **History** (opt. mention OCR/Dup) | Show filterable run log, FAILED-only Retry guarantee, and 1-sentence thesis. |

## Scene 1 (0:00 – 0:45) — Opening Hook, Live Architecture & Baseline Reset

**Say:** *"Accounts-payable teams reconcile thousands of PDF invoices against Purchase Orders. Manual review doesn't scale, while trusting an LLM directly with payment approval is dangerous. Zamp separates those concerns: a local LLM extracts structured evidence, and deterministic Python business rules evaluate policy v1.0 with an immutable audit trail."*

**Do:** Point out the sidebar footer (**Backend: Online — DB connected**, **LLM provider: llamacpp · qwen2.5:1.5b**). Then click **Reset data -> Confirm reset** in the top header.
**Point Out:** Green confirmation banner (*"Baseline restored — 3 sample invoices, 0 runs"*). Mention that Overview shows live SQL counts (no canned metrics) and that clicking Reset during an active run is guarded by **409 Conflict**.

## Scene 2 (0:45 – 2:10) — Happy Path Upload & Full Intelligence Walkthrough

**Do:** Click **Upload invoice** -> drop/select `syn_01_clean_approve.pdf` -> click **Upload and process**.
**Say while live timeline ticks (~15–18s):** *"The upload API validates the %PDF magic bytes, hashes SHA-256, saves under a random server filename, returns 202 Accepted, and commits real database events every stage: Document Processing via PyMuPDF finds digital text so OCR is SKIPPED; Ollama runs structured JSON extraction after closing the DB transaction; then deterministic Vendor Match, PO Match, Duplicate Check, Amount, Line Items, Tax, Decision, and Persistence execute."*

**Do:** Once completed, click **Open full invoice intelligence**. Briefly scroll down:
• **Extracted information:** Clean table (Welding electrodes 200 kg @ 250 + Safety goggles 100 pcs @ 100 = 60,000 INR) + expandable **Inspect extraction evidence** confidence chips.
• **Purchase order comparison & Validation results:** All 6 check cards show **PASS / MATCHED / NO_DUPLICATE_FOUND** against `PO-2044`.
• **Final decision:** Policy v1.0 returns **APPROVE** (`APPROVE_001`). *(Contingency: if 1.5B omits numeric confidence and routes to REVIEW, explain immediately that policy v1.0 requires >=0.80 evidence rather than guessing!)*

## Scene 3 (2:10 – 3:35) — Edge Case #1: Financial Overbilling & Line-Item Mismatch

**Say:** *"Happy paths are easy. Now let's test an edge case where an active, legitimate vendor submits a valid digital invoice that tries to overbill a Purchase Order."*

**Do:** Click **Upload invoice** -> upload `syn_03_overbilling.pdf` (Vendor `V-002 XYZ Metals`, `PO-2045` = INR 120,000, invoice = INR 130,500). Wait ~15s -> click **Open full invoice intelligence**.

**Point Out on Detail Screen:**

- Vendor match, PO match, duplicate check, and tax arithmetic (`130,500 = 130,500`) ALL pass cleanly.
- Yet **Amount validation** fails with **AMOUNT_001**: INR 130,500 exceeds remaining INR 120,000 PO balance by INR 10,500 (well beyond the INR 100.00 tolerance).
- Simultaneously, **Line-item validation** flags **LINEITEM_001**: *Steel channel 75mm* billed for **150 pcs vs 60 pcs on PO**, plus 2 missing PO lines.
- **Result:** Deterministic **REVIEW** — financial discrepancies are never auto-approved, and they aren't silently blind-rejected either; exact math is handed to a human analyst.

## Scene 4 (3:35 – 4:25) — Edge Case #2: Blocked Vendor Rule Precedence

**Say:** *"Next, what happens if an invoice's amounts, line items, and tax all reconcile perfectly, but procurement has blocked the vendor?"*

**Do:** Click **Upload invoice** -> upload `syn_02_blocked_vendor.pdf` (`V-004 Sharma Electrical`, `PO-2047` = INR 30,000). Wait ~13s -> highlight the **REJECT** outcome banner on the run/detail page.

**Point Out:** PO match, duplicate check, amount check, line items (50 LED panel lights @ INR 600 = INR 30,000), and tax ALL pass! However, exact vendor matching identifies `V-004 (BLOCKED)` and Phase A triggers **VENDOR_BLOCK_001**. Because precedence is strictly **REJECT > REVIEW > APPROVE**, no downstream pass can ever override a hard compliance block.

### Optional High-Impact Edge Cases (If You Have 30s Spare or Reviewer Asks)

- **Renamed Byte-for-Byte Duplicate (`syn_05_duplicate_copy.pdf`):** Identical PDF bytes to `syn_01` under a completely different filename. Stage 6 detects matching `file_sha256` (`exclude_invoice_id` excludes self) and triggers **REJECT** via `DOC_DUP_001`.
- **Scanned Image-Only PDF + Provisional PO Balance (`syn_04_scanned_ocr.pdf`):** Zero embedded text layer -> Stage 2 switches to **Tesseract OCR**, recovers full text, then catches that `PO-2046` already has an unapproved seeded invoice (`INV-2026-1044`) -> `AMOUNT_002` provisional warning -> **REVIEW**!

## Scene 5 (4:25 – 5:00) — Review Queue, Processing History & Closing Thesis

**Do:**

1. Click **Review Queue** in the sidebar — show `syn_03_overbilling.pdf` queued automatically for human inspection.
2. Click **History** (`Processing history`) — show status/decision filters, real execution durations (~13–18s), and explain that if Ollama were down, the run logs **FAILED** (technical failure, never a fake business decision) and exposes a one-click **Retry** button that preserves prior events.
3. Briefly glance back at **Overview** showing updated donut chart & live metrics.

**Closing Line (4:50 – 5:00):** *"AI reads the unstructured PDF; deterministic Decimal code enforces financial policy; every rule, evidence snippet, and stage event is persisted for full auditability — so uncertain or conflicting invoices never pay out silently."*

# 6. Test PDF Cheat Sheet, Recovery Playbook & Q&A Prep

## Synthetic & Demo Invoices Quick Reference (`backend/data/`)

| File Path (under backend/data/) | Target Entity | Scenario Exercised | Key Signals / Rules | Outcome |
|---|---|---|---|---|
| `synthetic_invoices/syn_01_clean_approve.pdf` | V-001 · PO-2044 INR 60,000.00 | Clean digital text PDF matching all PO-2044 lines & totals. | APPROVE_001 (or REVIEW if 1.5B drops numeric conf.) | **APPROVE** |
| `synthetic_invoices/syn_02_blocked_vendor.pdf` | V-004 · PO-2047 INR 30,000.00 | Blocked vendor despite clean math & open PO. | VENDOR_BLOCK_001 | **REJECT** |
| `synthetic_invoices/syn_03_overbilling.pdf` | V-002 · PO-2045 INR 130,500.00 | Exceeds INR 120k PO by INR 10.5k + qty 150 vs 60 + 2 missing PO lines. | AMOUNT_001, LINEITEM_001 | **REVIEW** |
| `synthetic_invoices/syn_04_scanned_ocr.pdf` | V-005 · PO-2046 INR 15,000.00 | 200-DPI rasterized image PDF (0 embedded chars) -> Tesseract OCR + provisional PO. | OCR COMPLETED, AMOUNT_002, LINEITEM_001 | **REVIEW** |
| `synthetic_invoices/syn_05_duplicate_copy.pdf` | V-001 · PO-2044 INR 60,000.00 | Exact SHA-256 copy of syn_01 under new filename. | DOC_DUP_001 (plus AMOUNT_001 if syn_01 approved) | **REJECT** |
| `demo_invoices/demo_overbilling.pdf` | V-002 · PO-2045 INR 116,820.00 | Alternate fixture testing overbilling when prior invoice exists on PO-2045. | AMOUNT_001 / AMOUNT_002 | **REVIEW** |
| `demo_invoices/demo_no_po_ref.pdf` | No PO # printed | Tests candidate PO matching without PO reference. | PO_008 / PO_001 | **REVIEW** |

## Live Demo Contingency & Recovery Guide

| Situation | Exact Action & Technical Explanation |
|---|---|
| **Run ends FAILED (LLM timeout / glitch)** | Show the red failure panel on the run page! Explain that failures never pollute business metrics, then click **Retry this run** to re-queue atomically while keeping full history. |
| **409 Conflict when clicking Reset data** | A run is still `PENDING` or `RUNNING` (e.g., an upload is still inferring). Wait ~15s for completion, click **Reset data -> Confirm reset** again. |
| **401 Unauthorized after restart / fresh browser** | Auth guard requires the session cookie. Enter `ZampPhase9Demo!` on the sign-in screen. |
| **syn_01 routes to REVIEW instead of APPROVE** | Open **Extracted information -> Inspect extraction evidence**: if the 1.5B model wrote text without numeric `confidence >= 0.80`, point out that `decision_policy.py` treats un-scored extraction conservatively as `REVIEW`. |

## Likely Reviewer Q&A (Concise Answers to Memorize)

**Q1: Why not let the LLM decide APPROVE / REVIEW / REJECT?** — LLMs are non-deterministic and bad at cumulative PO balance arithmetic across past DB rows. By restricting the LLM to schema extraction and enforcing rules in pure Python (`decide()`), policy v1.0 is unit-testable, reproducible, and auditable.

**Q2: How do you prevent double-billing when two invoices hit the same PO concurrently?** — Stage 11 (`_stage_persistence`) takes a SQLite `RESERVED` write lock on the PO row and re-runs `validate_amount()` right before committing an `APPROVE`, downgrading stale approvals via `AMOUNT_003`.

**Q3: What counts toward PO committed balance, and what is 'provisional balance'?** — Only invoices with a completed `APPROVE` run reduce committed PO balance (counted per invoice so retries never double-count). If unapproved invoices exist with zero `APPROVE` history, balance basis becomes **provisional** (`AMOUNT_002`) so the system never silently auto-approves.

**Q4: Why didn't a blocked vendor fuzzy-match trigger REJECT?** — `vendor_matcher.py` explicitly excludes `BLOCKED` vendors from the fuzzy pool (`Vendor.status != VendorStatus.BLOCKED`). Only an unambiguous Tax ID or exact normalized name triggers hard `VENDOR_BLOCK_001`; uncertain matches go to `REVIEW`.

**Q5: How do you protect uploaded files and local execution?** — Files must pass extension, 20 MB cap, and `%PDF-` magic bytes; filenames are never used as paths (stored as `<hex>_<sha16>.pdf`), served strictly confined under `UPLOAD_DIR`, and write routes enforce session cookies + loopback/token checks + rate limits.